

第 10 章：故障恢复

2026 版

邹兆年

哈尔滨工业大学
计算学部
海量数据计算研究中心
电子邮件: znzou@hit.edu.cn

2026 春

教学内容¹

- ① 10.1 故障
- ② 10.2 缓冲池策略
- ③ 10.3 预写式日志
 - 10.3.1 预写式日志的基本概念
 - 10.3.1 基于 Undo 日志的故障恢复
 - 10.3.2 基于 Redo 日志的故障恢复
 - 10.3.1 基于 Undo/Redo 日志的故障恢复
- ④ 10.4 检查点

¹课件更新于 2026 年 6 月 16 日

10.1 故障

故障的类型

- 事务故障 (Transaction Failure)
- 系统故障 (System Failure)
- 存储介质故障 (Storage Media Failure)

事务故障 (Transaction Failure)

逻辑错误 (Logical Error)

- 事务因内部错误 (Internal Error) 而无法完成, 如事务 Bug、违反完整性约束等

内部状态错误 (Internal State Error)

- DBMS 因内部状态错误 (如死锁) 而必须中止活跃事务

系统故障 (System Failure)

软件故障 (Software Failure)

- DBMS 的 Bug 导致的故障

硬件故障 (Hardware Failure)

- 运行 DBMS 的计算机崩溃 (Crash), 如断电
- 假设崩溃不会损坏非易失存储器中的数据

存储介质故障 (Storage Media Failure)

非易失存储器发生故障, 损坏了存储的数据

- 假设数据损坏可以被检测出来, 如使用校验和 (Checksum)
- 任何 DBMS 都无法从这种故障中恢复, 必须从备份中还原

10.2 缓冲池策略

故障恢复的基本操作

故障恢复时，DBMS 只会执行两种基本操作

撤销 (Undo) 修改

撤销不完整事务 (Incomplete Txn) 对数据库的修改

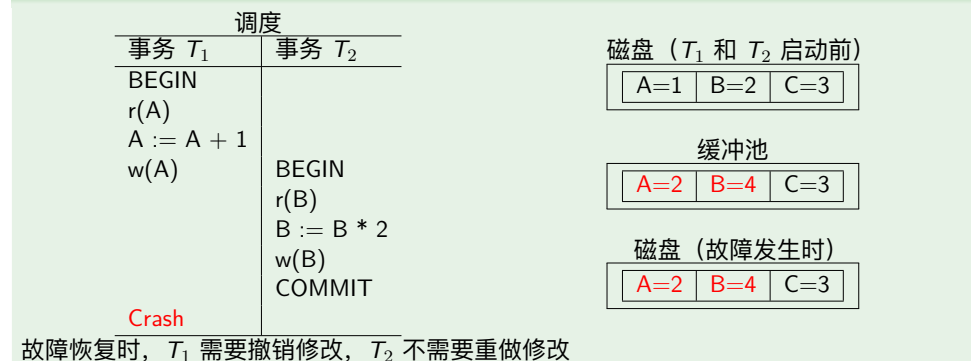
重做 (Redo) 修改

重新执行已提交事务 (Committed Txn) 对数据库的修改

能否仅撤销 (Undo) 修改?

如果可以保证已提交的事务对数据库的修改全部写入磁盘，则 DBMS 在故障恢复时只需撤销修改

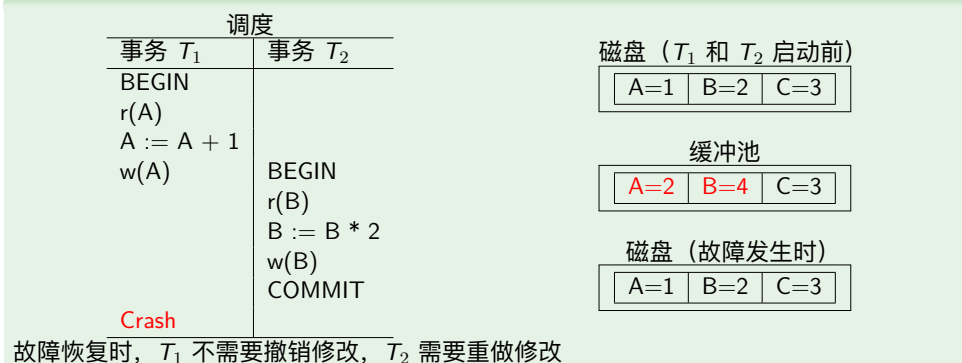
例 (仅撤销修改)



能否仅重做 (Redo) 修改?

如果可以保证不完整的事务对数据库的修改全都不会写入磁盘，则 DBMS 在故障恢复时只需重做修改

例 (仅重做修改)

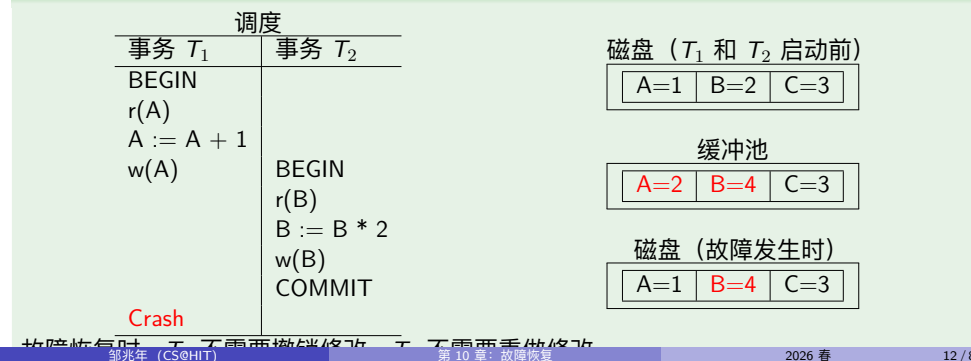


能否既不撤销 (Undo) 修改，也不重做 (Redo) 修改?

如果满足下面两个条件，则 DBMS 在故障恢复时既不需要撤销修改，也不需要重做修改

- 可以保证已提交的事务对数据库的修改全部写入磁盘
- 可以保证不完整的事务对数据库的修改全都不会写入磁盘

例 (既不撤销修改，也不重做修改)



什么情况下，既要撤销（Undo）修改，又要重做（Redo）修改？

如果事务对数据库的修改是否写入磁盘无任何保证，则 DBMS 在故障恢复时，既要撤销修改，又要重做修改

例 (既要撤销修改，又要重做修改)

调度		磁盘 (T_1 和 T_2 启动前)
事务 T_1	事务 T_2	
BEGIN		A=1 B=2 C=3
r(A)		
A := A + 1		
w(A)		
	BEGIN	
	r(B)	
	B := B * 2	
	w(B)	
	COMMIT	
Crash		

故障恢复时， T_1 需要撤销修改， T_2 需要重做修改

邹兆年 (CS@HIT)

第 10 章：故障恢复

2026 春

13 / 83

缓冲池策略 (Buffer Pool Policy)

故障恢复时要不要撤销/重做修改取决于 DBMS 的缓冲池策略

STEAL 

FORCE 

邹兆年 (CS@HIT)

第 10 章：故障恢复

2026 春

14 / 83

STEAL 策略

STEAL  DBMS 允许将未提交的事务对数据库的修改写入磁盘，覆盖现有数据

例 (STEAL 策略)

调度		磁盘 (T_1 和 T_2 启动前)
事务 T_1	事务 T_2	
BEGIN		A=1 B=2 C=3
r(A)		
A := A + 1		
w(A)		
	BEGIN	
	r(B)	
	B := B * 2	
	w(B)	
	COMMIT	
Crash		

故障恢复时， T_1 需要撤销修改， T_2 需要重做修改

邹兆年 (CS@HIT)

第 10 章：故障恢复

2026 春

15 / 83

STEAL 策略

- 运行时：缓冲池效率高
- 故障恢复时：必须撤销不完整事务的修改

例 (STEAL 策略)

调度		磁盘 (T_1 和 T_2 启动前)
事务 T_1	事务 T_2	
BEGIN		A=1 B=2 C=3
r(A)		
A := A + 1		
w(A)		
	BEGIN	
	r(B)	
	B := B * 2	
	w(B)	
	COMMIT	
Crash		

故障恢复时， T_1 需要撤销修改， T_2 需要重做修改

邹兆年 (CS@HIT)

第 10 章：故障恢复

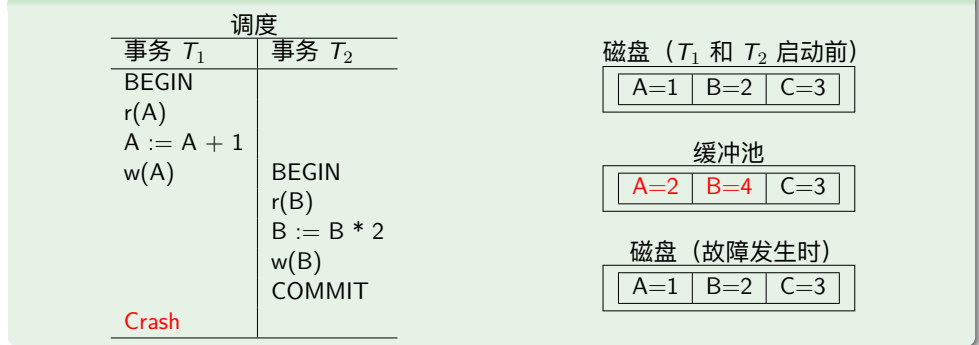
2026 春

15 / 83

NO-STEAL 策略

STEAL ☐ DBMS 不允许将未提交的事务对数据库的修改写入磁盘，覆盖现有数据

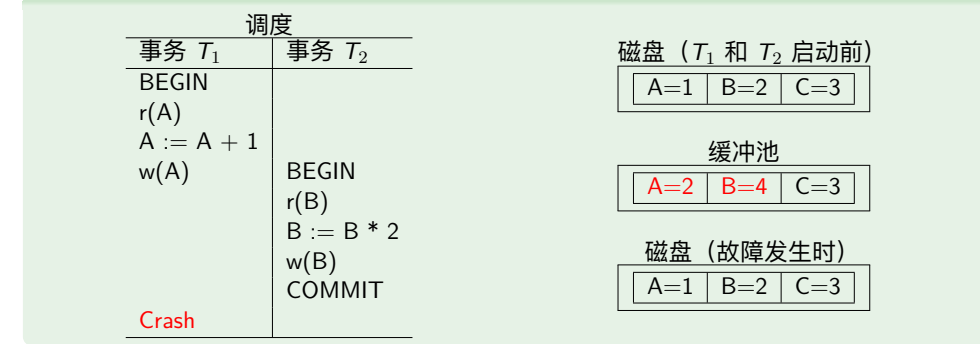
例 (NO-STEAL 策略)



NO-STEAL 策略

- 运行时：缓冲池效率低
- 故障恢复时：不需要撤销不完整事务的修改

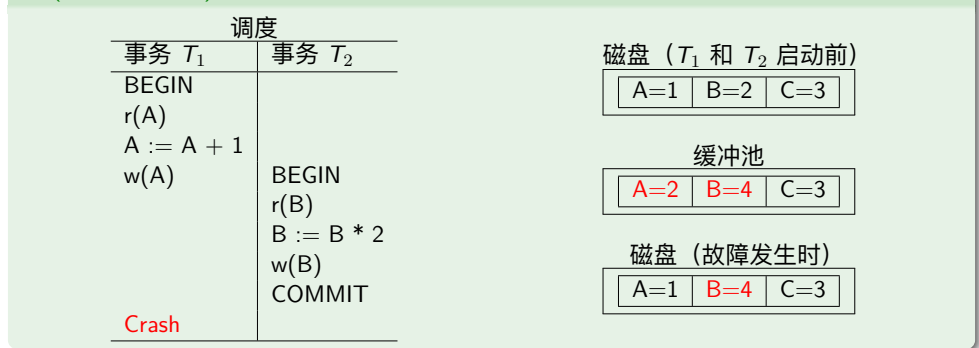
例 (NO-STEAL 策略)



FORCE 策略

FORCE ☒ DBMS 强制事务在提交前必须将其所做修改全部写入磁盘

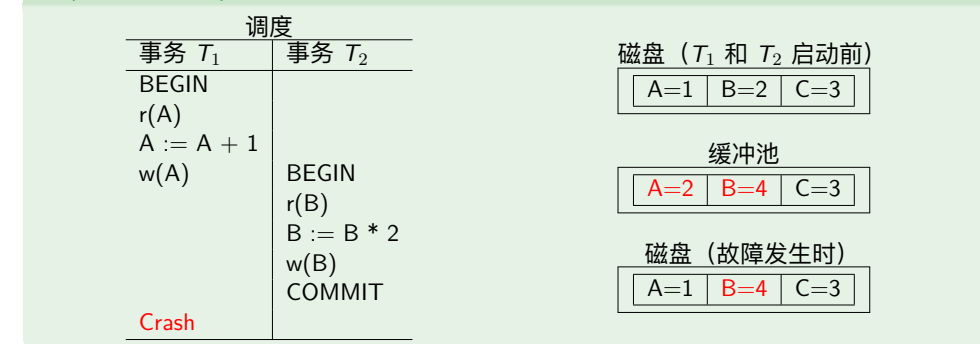
例 (FORCE 策略)



FORCE 策略

- 特点：I/O 效率低
- 故障恢复时：不需要重做已提交事务的修改

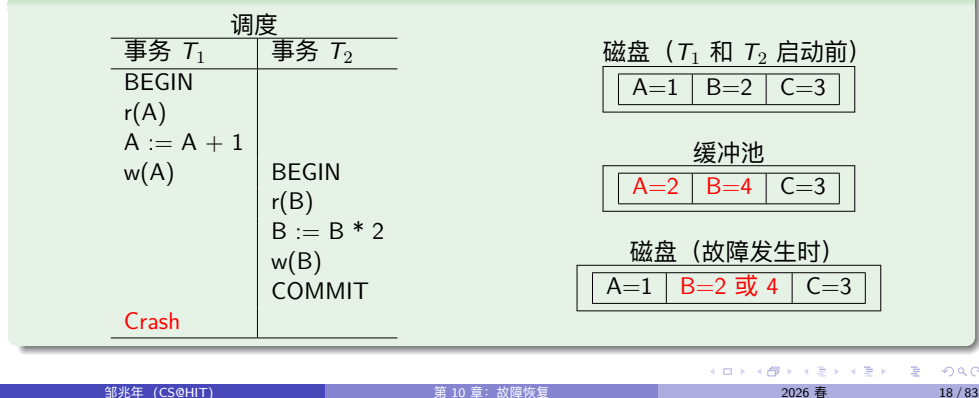
例 (FORCE 策略)



NO-FORCE 策略

FORCE  DBMS 不强制事务在提交前必须将其所做修改全部写入磁盘

例 (NO-FORCE 策略)



邹兆年 (CS@HIT)

第 10 章: 故障恢复

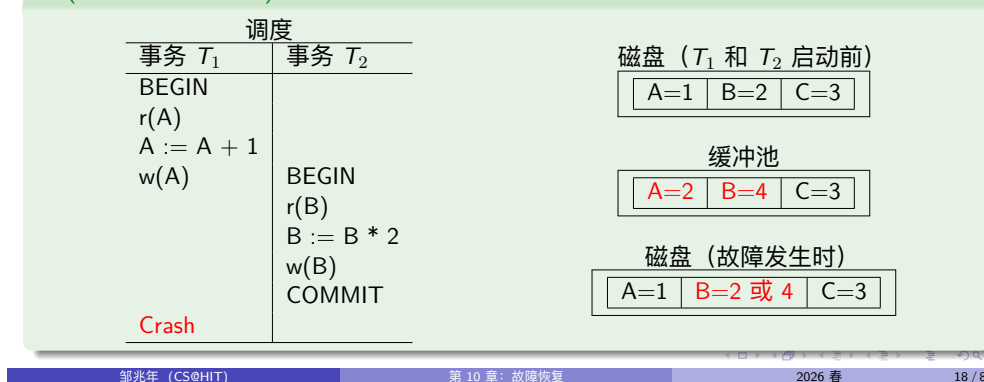
2026 春

18 / 83

NO-FORCE 策略

- 特点: I/O 效率高
- 故障恢复时: 必须重做已提交事务的修改

例 (NO-FORCE 策略)



邹兆年 (CS@HIT)

第 10 章: 故障恢复

2026 春

18 / 83

缓冲池策略 (Buffer Pool Policy)

	I/O 效率低	I/O 效率高
缓冲池效率高	STEAL + FORCE	STEAL + NO-FORCE
缓冲池效率低	NO-STEAL + FORCE	NO-STEAL + NO-FORCE

邹兆年 (CS@HIT)

第 10 章: 故障恢复

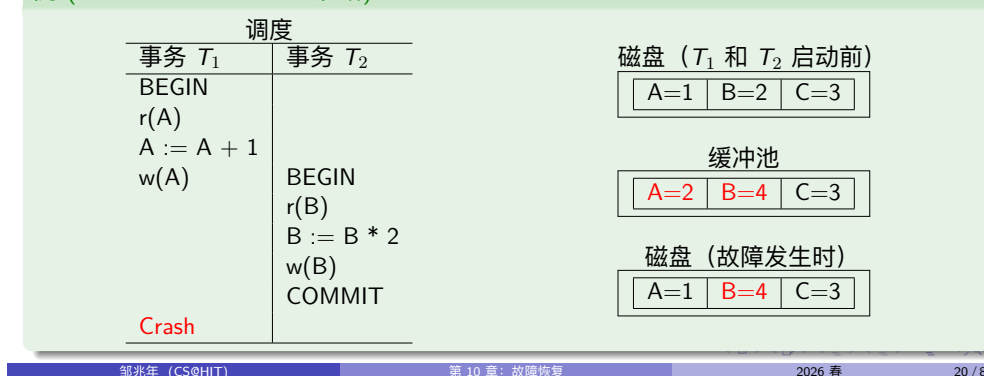
2026 春

19 / 83

NO-STEAL+FORCE 策略

- STEAL  未提交事务不可能将其修改写回磁盘 $\Rightarrow$ 无需撤销修改
- FORCE  已提交事务已将修改全部写回磁盘 $\Rightarrow$ 无需重做修改

例 (NO-STEAL+FORCE 策略)



邹兆年 (CS@HIT)

第 10 章: 故障恢复

2026 春

20 / 83

NO-STEAL+FORCE 策略

- 优点: 实现简单
- 缺点: 缓冲池得能存得下所有未提交事务所做的修改

例 (NO-STEAL+FORCE 策略)

调度		
事务 T_1	事务 T_2	
BEGIN		磁盘 (T_1 和 T_2 启动前)
$r(A)$		A=1 B=2 C=3
$A := A + 1$		
$w(A)$		缓冲池
	BEGIN	A=2 B=4 C=3
	$r(B)$	
	$B := B * 2$	磁盘 (故障发生时)
	$w(B)$	A=1 B=4 C=3
	COMMIT	
Crash		

10.3 预写式日志

预写式日志 (Write-Ahead Log, WAL)

除了数据文件以外, DBMS 还维护一个日志文件, 用于记录事务对数据库的修改

- 假定日志文件存储在稳定存储器 (Stable Storage) 中
- 日志记录 (Log Record) 包含撤销或重做修改时所需的信息

在将修改过的对象写到磁盘之前, 必须先将修改此对象的日志记录刷写到磁盘

10.3.1 预写式日志的基本概念

WAL 协议 (WAL Protocol)

`<tid, BEGIN>`

事务启动日志记录

事务 T 启动时, 向日志末尾写入记录 `<tid, BEGIN>`

- `tid`: T 的 ID

WAL 协议 (WAL Protocol)

`<tid, COMMIT>`

事务提交日志记录

事务 T 提交时, 向日志末尾写入记录 `<tid, COMMIT>`

- `tid`: T 的 ID
- 在 DBMS 向应用程序返回 T 提交成功的响应之前, 必须保证 T 的所有日志记录都已刷写到磁盘
- `<tid, COMMIT>` 写入磁盘才代表事务提交

WAL 协议 (WAL Protocol)

`<tid, oid, before, after>`

事务修改日志记录

事务 T 修改对象 A 时, 向日志末尾写入记录 `<tid, oid, before, after>`

- `tid`: T 的 ID
- `oid`: A 的 ID
- `before`: A 修改前的值 (撤销修改时使用)
- `after`: A 修改后的值 (重做修改时使用)

WAL 协议 (WAL Protocol)

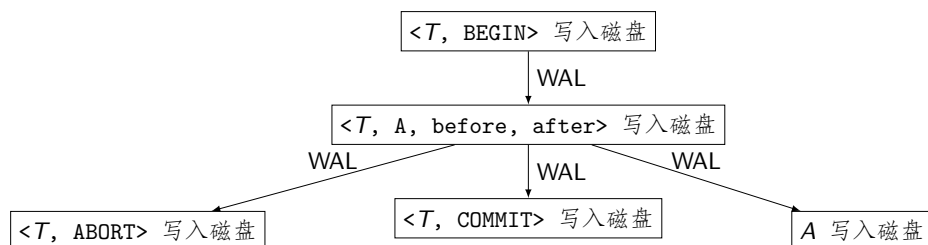
`<tid, ABORT>`

事务中止日志记录

事务 T 完成回滚后, 向日志末尾写入记录 `<tid, ABORT>`

- `tid`: T 的 ID

WAL 日志记录写入磁盘的顺序



基于 WAL 的故障恢复

第 1 部分: 事务正常执行时的行为

- 记录日志
- 按照缓冲池策略将修改过的对象写到磁盘

第 2 部分: 故障恢复时的行为

- 根据日志和缓冲池策略, 撤销或重做事务的修改

事务的分类

根据日志将事务分为 3 类

- 已提交事务 (Committed Txn)
- 已中止事务 (Aborted Txn)
- 不完整事务 (Incomplete Txn)

已提交事务 (Committed Txn)

定义 (已提交事务)

- 既有 `<T, BEGIN>`, 又有 `<T, COMMIT>`

```
...
<T, BEGIN>
...
<T, COMMIT>
...
```

已中止事务 (Aborted Txn)

定义 (已中止事务)

- 既有 `<T, BEGIN>`, 又有 `<T, ABORT>`
- 在事务正常执行和故障恢复过程中, 如果 T 的修改已全部撤销, 则将日志记录 `<T, ABORT>` 写到日志末尾
- 已中止事务相当于从未执行过, 故不需要撤销修改, 更不需要重做修改

```
...  
<T, BEGIN>  
...  
<T, ABORT>  
...
```

不完整事务 (Incomplete Txn)

定义 (不完整事务)

- 只有 `<T, BEGIN>`, 而没有 `<T, COMMIT>` 或 `<T, ABORT>`

```
...  
<T, BEGIN>  
...
```

故障恢复时的行为

已提交事务

- 如果一个已提交事务的修改已全部写入磁盘, 则无需重做其修改
- 否则, 需要重做其修改

不完整事务

- 如果一个不完整事务的任何修改都未写入磁盘, 则无需撤销其修改
- 否则, 需要撤销其修改

缓冲池策略决定了上述行为

基于 WAL 的故障恢复技术的分类

根据缓冲池策略的不同, 分为三类:

- **Undo Logging:** WAL + STEAL + FORCE
- **Redo Logging:** WAL + NO-STEAL + NO-FORCE
- **Redo+Undo Logging:** WAL + STEAL + NO-FORCE

10.3.1 基于 Undo 日志的故障恢复

基于 Undo 日志的故障恢复

STEAL 

FORCE 

	I/O 效率低	I/O 效率高
缓冲池效率高	STEAL + FORCE	STEAL + NO-FORCE
缓冲池效率低	NO-STEAL + FORCE	NO-STEAL + NO-FORCE

STEAL+FORCE 策略对故障恢复的影响

- STEAL  不完整事务可能将其修改写回磁盘 $\Rightarrow$ 必须撤销
- FORCE  已提交事务已将修改全部写回磁盘 $\Rightarrow$ 无需重做

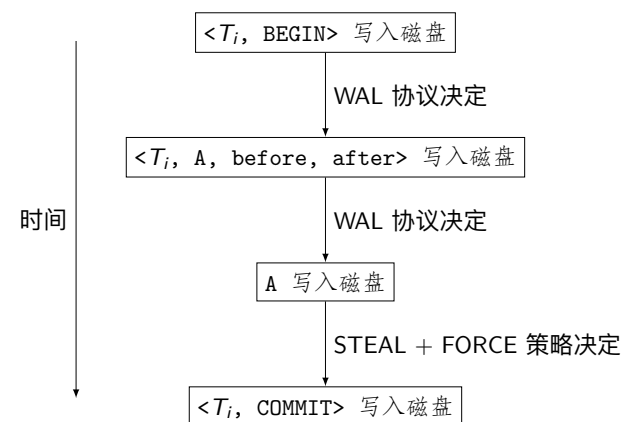
例 (STEAL+FORCE 策略)

调度		磁盘 (T_1 和 T_2 启动前)
事务 T_1	事务 T_2	
BEGIN		A=1 B=2 C=3
r(A)		
A := A + 1		
w(A)		
	BEGIN	
	r(B)	
	B := B * 2	
	w(B)	
	COMMIT	
Crash		

缓冲池		
A=2	B=4	C=3

磁盘 (故障发生时)		
A=2	B=4	C=3

Undo 日志记录写入磁盘的顺序



记录 Undo 日志

例 (记录 Undo 日志)

Step	Action	t	M _A	M _B	D _A	D _B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	t := t * 2	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	t := t * 2	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	FLUSH LOG						
9	OUTPUT(A)	16	16	16	16	8	
10	OUTPUT(B)	16	16	16	16	16	
11	COMMIT						<T, COMMIT>
12	FLUSH LOG						

基于 Undo 日志的故障恢复过程

- 1 确定事务的类型（已提交事务、已中止事务、不完整事务）
- 2 撤销不完整事务

确定事务的类型

扫描日志（从前向后或从后向前都可以）

- 如果日志记录是 <T, COMMIT>，则 T 是已提交事务，记录下来
- 如果日志记录是 <T, ABORT>，则 T 是已中止事务，记录下来

日志扫描结束后，未被记录为已提交或已中止的事务一定是不完整事务

撤销不完整事务

从后向前扫描日志，对于不完整事务 T，

- 如果日志记录是 <T, A, before, after>，则将磁盘上 A 的值写为 before
- 如果日志记录是 <T, BEGIN>，则 T 撤销完毕。为了在以后的故障恢复中不再撤销 T，向日志末尾写入 <T, ABORT>

基于 Undo 日志的故障恢复方法

确定事务的类型和撤销不完整事务可以合并，通过一遍日志扫描完成

基于 Undo 日志的故障恢复算法

从后向前扫描日志一遍，根据每条日志记录的类型执行对应的动作

- $\langle T, \text{COMMIT} \rangle$: 将 T 记录为已提交事务（无需重做）
- $\langle T, \text{ABORT} \rangle$: 将 T 记录为已中止事务（无需撤销，也无需重做）
- $\langle T, A, \text{before}, \text{after} \rangle$: 如果 T 既不是已提交事务，也不是已中止事务，则将磁盘上 A 的值写为 before
- $\langle T, \text{BEGIN} \rangle$: 如果 T 既不是已提交事务，也不是已中止事务，则 T 撤销完毕；为了在以后的故障恢复中不再撤销 T ，向日志末尾写入 $\langle T, \text{ABORT} \rangle$

基于 Undo 日志的故障恢复

例 (基于 Undo 日志的故障恢复)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							$\langle T, \text{BEGIN} \rangle$
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	$\langle T, A, 8, 16 \rangle$
5	READ(B, t)	8	16	8	8	8	

- T 是不完整事务，必须撤销，将 A 恢复为 8
- 向日志末尾写入 $\langle T, \text{ABORT} \rangle$

基于 Undo 日志的故障恢复

例 (基于 Undo 日志的故障恢复)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							$\langle T, \text{BEGIN} \rangle$
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	$\langle T, A, 8, 16 \rangle$
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	$\langle T, B, 8, 16 \rangle$
8	FLUSH LOG						
9	OUTPUT(A)	16	16	16	16	8	
10	OUTPUT(B)	16	16	16	16	16	

- T 是不完整事务，必须撤销，先将 B 恢复为 8，后将 A 恢复为 8
- 向日志末尾写入 $\langle T, \text{ABORT} \rangle$

基于 Undo 日志的故障恢复

例 (基于 Undo 日志的故障恢复)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							$\langle T, \text{BEGIN} \rangle$
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	$\langle T, A, 8, 16 \rangle$
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	$\langle T, B, 8, 16 \rangle$
8	FLUSH LOG						
9	OUTPUT(A)	16	16	16	16	8	
10	OUTPUT(B)	16	16	16	16	16	
11	COMMIT						$\langle T, \text{COMMIT} \rangle$
12	FLUSH LOG						

- T 是已提交事务，无需重做

10.3.2 基于 Redo 日志的故障恢复

基于 Redo 日志的故障恢复

STEAL  OFF

FORCE  OFF

	I/O 效率低	I/O 效率高
缓冲池效率高	STEAL + FORCE	STEAL + NO-FORCE
缓冲池效率低	NO-STEAL + FORCE	NO-STEAL + NO-FORCE

NO-STEAL+NO-FORCE 策略对故障恢复的影响

- STEAL  OFF 不完整事务不可能将其修改写回磁盘 $\Rightarrow$ 无需撤销
- FORCE  OFF 已提交事务可能未将修改全部写回磁盘 $\Rightarrow$ 必须重做

例 (NO-STEAL+NO-FORCE 策略)

调度		磁盘 (T_1 和 T_2 启动前)			
事务 T_1	事务 T_2				
BEGIN		<table border="1"><tr><td>A=1</td><td>B=2</td><td>C=3</td></tr></table>	A=1	B=2	C=3
A=1	B=2	C=3			
$r(A)$					
$A := A + 1$					
$w(A)$					
	BEGIN				
	$r(B)$				
	$B := B * 2$				
	$w(B)$				
	COMMIT				
Crash					

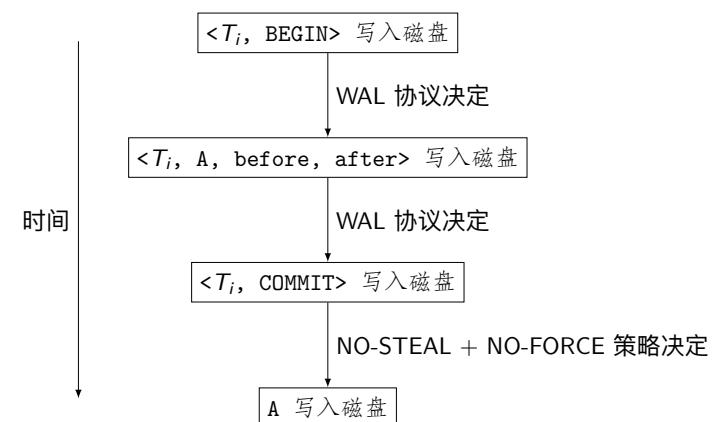
缓冲池

A=2	B=4	C=3
-----	-----	-----

磁盘 (故障发生时)

A=1	B=2	C=3
-----	-----	-----

Redo 日志记录写入磁盘的顺序



记录 Redo 日志

例 (记录 Redo 日志)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	COMMIT						<T, COMMIT>
9	FLUSH LOG						
10	OUTPUT(A)	16	16	16	16	8	
11	OUTPUT(B)	16	16	16	16	16	

基于 Redo 日志的故障恢复过程

- 1 确定事务的类型 (已提交事务、已中止事务、不完整事务)
- 2 重做已提交事务

重做已提交事务

从前向后扫描日志, 对于已提交事务 T ,

- 如果日志记录是 <T, BEGIN>, 则跳过
- 如果日志记录是 <T, A, before, after>, 则将磁盘上 A 的值写为 after
- 如果日志记录是 <T, COMMIT>, 则 T 重做完毕, 不需要再做什么

基于 Redo 日志的故障恢复方法

确定事务的类型和重做已提交事务不可以合并, 通过两遍日志扫描完成

基于 Redo 日志的故障恢复算法

扫描日志一遍 (从前向后或从后向前都可以), 根据每条日志记录的类型执行对应的动作

- <T, COMMIT>: 将 T 记录为已提交事务 (必须重做)
- <T, ABORT>: 将 T 记录为已中止事务 (无需撤销, 也无需重做)
- 其他类型日志记录: 跳过

从前向后扫描日志一遍, 根据每条日志记录的类型执行对应的动作

- <T, A, before, after>: 如果 T 是已提交事务, 则将磁盘上 A 的值写为 after
- 其他类型日志记录: 跳过

基于 Redo 日志的故障恢复

例 (基于 Redo 日志的故障恢复)

Step	Action	t	M _A	M _B	D _A	D _B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	t := t * 2	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	t := t * 2	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>

- T 是不完整事务，无需撤销

基于 Redo 日志的故障恢复

例 (基于 Redo 日志的故障恢复)

Step	Action	t	M _A	M _B	D _A	D _B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	t := t * 2	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	t := t * 2	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	COMMIT						<T, COMMIT>
9	FLUSH LOG						

- T 是已提交事务，必须重做，先将 A 写为 16，后将 B 写为 16

基于 Redo 日志的故障恢复

例 (基于 Redo 日志的故障恢复)

Step	Action	t	M _A	M _B	D _A	D _B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	t := t * 2	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	t := t * 2	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	COMMIT						<T, COMMIT>
9	FLUSH LOG						
10	OUTPUT(A)	16	16	16	16	8	
11	OUTPUT(B)	16	16	16	16	16	

- T 是已提交事务，必须重做，先将 A 写为 16，后将 B 写为 16

10.3.1 基于 Undo/Redo 日志的故障恢复

基于 Undo/Redo 日志的故障恢复

STEAL 

FORCE 

	I/O 效率低	I/O 效率高
缓冲池效率高	STEAL + FORCE	STEAL + NO-FORCE
缓冲池效率低	NO-STEAL + FORCE	NO-STEAL + NO-FORCE

Navigation icons

STEAL+NO-FORCE 策略对故障恢复的影响

- STEAL  不完整事务可能将其修改写回磁盘 $\Rightarrow$ 必须撤销
- FORCE  已提交事务可能未将修改全部写回磁盘 $\Rightarrow$ 必须重做

例 (STEAL+NO-FORCE 策略)

调度	
事务 T_1	事务 T_2
BEGIN	
$r(A)$	
$A := A + 1$	
$w(A)$	
	BEGIN
	$r(B)$
	$B := B * 2$
	$w(B)$
	COMMIT
Crash	

磁盘 (T_1 和 T_2 启动前)

A=1	B=2	C=3
-----	-----	-----

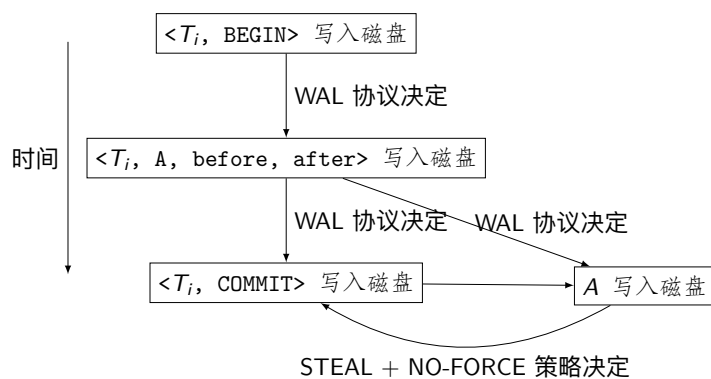
缓冲池

A=2	B=4	C=3
-----	-----	-----

磁盘 (故障发生时)

A=2	B=2	C=3
-----	-----	-----

Undo/Redo 日志记录写入磁盘的顺序



Navigation icons

记录 Undo/Redo 日志

例 (Redo+Undo Logging)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							$\langle T, \text{BEGIN} \rangle$
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	$\langle T, A, 8, 16 \rangle$
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	$\langle T, B, 8, 16 \rangle$
8	OUTPUT(A)	16	16	16	16	8	
9	COMMIT						$\langle T, \text{COMMIT} \rangle$
10	FLUSH LOG						
11	OUTPUT(B)	16	16	16	16	16	

Navigation icons

基于 Undo/Redo 日志的故障恢复过程

- 1 确定事务的类型（已提交事务、已中止事务、不完整事务）
 - 2 重做已提交事务
 - 3 撤销不完整事务
- 2 和 3 的顺序无所谓，可以按 1, 3, 2 的顺序执行，这样 1 和 3 可以合并

基于 Undo/Redo 日志的故障恢复方法

基于 Undo/Redo 日志的故障恢复算法

- 1 执行基于 Undo 日志的故障恢复方法来确定事务的类型及撤销不完整事务
- 2 执行基于 Redo 日志的故障恢复方法来重做已提交事务，但不用再确定事务的类型

基于 Undo/Redo 日志的故障恢复

例 (基于 Undo/Redo 日志的故障恢复)

Step	Action	t	M _A	M _B	D _A	D _B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	t := t * 2	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	t := t * 2	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	OUTPUT(A)	16	16	16	16	8	

- T 是不完整事务，必须撤销，先将 B 写为 8，后将 A 写为 8

基于 Undo/Redo 日志的故障恢复

例 (基于 Undo/Redo 日志的故障恢复)

Step	Action	t	M _A	M _B	D _A	D _B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	t := t * 2	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	t := t * 2	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	OUTPUT(A)	16	16	16	16	8	
9	COMMIT						<T, COMMIT>
10	FLUSH LOG						
11	OUTPUT(B)	16	16	16	16	16	

- T 是已提交事务，必须重做，先将 A 写为 16，后将 B 写为 16

缓冲池策略的比较

运行时效率		
	FORCE	NO-FORCE
STEAL	—	最快
NO-STEAL	最慢	—

故障恢复效率		
	FORCE	NO-FORCE
STEAL	—	最慢
NO-STEAL	最快	—

几乎所有 DBMS 都采用 STEAL+NO-FORCE 策略

10.4 检查点

组提交 (Group Commit)

每条日志记录单独刷写到磁盘的 I/O 开销太大

在内存中设置日志缓冲区 (Log Buffer)，将日志记录写到日志缓冲区，然后成批刷写到日志文件

- 日志缓冲区满时刷写
- 定时刷写

WAL 日志的问题

- 日志文件越来越大
- 故障恢复需要扫描日志文件 1~2 遍，会变得越来越慢

WAL 日志记录有必要全保留吗?

例 (WAL)

使用基于 Undo 日志的故障恢复

```
<T1, BEGIN>
<T1, A, 5, 15>
<T2, BEGIN>
<T2, B, 10, 20>
<T2, C, 15, 25>
<T2, COMMIT> 这之前日志记录记载的修改一定已写入磁盘
<T3, BEGIN> 故障恢复时, 扫描到这里即可
<T1, D, 20, 30>
<T3, E, 25, 35>
<T1, COMMIT>
<T3, F, 30, 40>
```

这种天然的位置不好找, 也不一定存在, 需要人为设置这样的位置

检查点 (Checkpoint)

DBMS 主动设置检查点

- 在日志中设置特定的位置, 保证之前的日志记录对故障恢复都没用
- 后续故障恢复只需扫描到最新的检查点位置

简单检查点 (Simple Checkpoint)

简单检查点

- 启动检查点, 停止启动新事务, 活跃事务继续执行
- 将日志刷写入磁盘
- 根据缓冲池策略, 将脏页 (Dirty Page) 写到磁盘
- 待活跃事务全部结束 (提交或中止), 日志和脏页全部写入磁盘后, 向日志末尾写入日志记录 <CHECKPOINT>

后续故障恢复只需扫描到最近的检查点日志记录 <CHECKPOINT>

简单检查点

例 (简单检查点)

```
<T1, BEGIN>
<T1, A, 5, 15>
<T2, BEGIN>
<T2, B, 10, 20>
<T2, C, 15, 25>
<T2, COMMIT>
<T1, D, 20, 30>
<T1, COMMIT>
<CHECKPOINT>
```

启动检查点, 停止启动新事务, 活跃事务 T_1 和 T_2 继续执行

向数据文件中写入修改 $A \leftarrow 15, B \leftarrow 20, C \leftarrow 25, D \leftarrow 30$
等待 T_1 和 T_2 结束

简单检查点的缺点

- 停止启动新事务
- 等待活跃事务全部结束

模糊检查点 (Fuzzy Checkpoint)

检查点过程中允许新事务启动

检查点开始

向日志末尾写入 $\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$

- $T_1, T_2, \dots, T_n$ 是检查点开始时的活跃事务

检查点中间

根据缓冲池策略, 将脏页写入磁盘

- 如果采用 STEAL 策略, 则将全部脏页写入磁盘
- 否则, 只将脏页中已提交事务所做的修改写入磁盘

检查点结束

向日志末尾写入 $\langle \text{END CHECKPOINT} \rangle$, 并将日志刷写到磁盘

- 如果采用 NO-FORCE 策略, 则写完全部脏页后即可结束检查点
- 否则, 在 $T_1, T_2, \dots, T_n$ 全部提交后, 才能结束检查点

涉及检查点的故障恢复

仅介绍 STEAL + NO-FORCE 策略 (自行思考其它策略下的恢复方法)

Redo 阶段: 重做已提交事务

- 从前向后扫描日志
- 从哪条日志记录开始?

Undo 阶段: 撤销不完整事务

- 从后向前扫描日志
- 到哪条日志记录为止?

Redo 阶段

日志中最近的完整检查点

$\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$

...

$\langle \text{END CHECKPOINT} \rangle$

定理

如果事务 T 启动早于 $\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$, 并且故障恢复时必须重做 T , 则 $T \in \{T_1, T_2, \dots, T_n\}$

- 从日志记录 $\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$ 开始向后扫描日志
- 不需要从 $T_1, T_2, \dots, T_n$ 中已提交事务的最早启动日志记录 $\langle T_i, \text{BEGIN} \rangle$ 开始扫描

证明 I

情况 1

日志记录	事实
$\langle T, \text{BEGIN} \rangle$	
...	
$\langle T, \text{COMMIT} \rangle$	
...	
$\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$	$T \notin \{T_1, T_2, \dots, T_n\}$
...	
$\langle \text{END CHECKPOINT} \rangle$	T 所做的修改已全部写入磁盘, 无需重做

证明 II

情况 2

日志记录	事实
$\langle T, \text{BEGIN} \rangle$	
...	
$\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$	$T \in \{T_1, T_2, \dots, T_n\}$
...	
$\langle T, \text{COMMIT} \rangle$	
...	
$\langle \text{END CHECKPOINT} \rangle$	T 所做的修改已全部写到磁盘, 无需重做

证明 III

情况 3

日志记录	事实
$\langle T, \text{BEGIN} \rangle$	
...	
$\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$	$T \in \{T_1, T_2, \dots, T_n\}$
...	
$\langle \text{END CHECKPOINT} \rangle$	
...	
$\langle T, \text{COMMIT} \rangle$	T 所做的修改未必全部写入磁盘, 必须重做

Undo 阶段

日志中最近的完整检查点

$\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$
...
 $\langle \text{END CHECKPOINT} \rangle$

定理

如果事务 T 启动早于 $\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$, 并且故障恢复时必须撤销 T , 则 $T \in \{T_1, T_2, \dots, T_n\}$

扫描到 $T_1, T_2, \dots, T_n$ 中不完整事务的最早启动日志记录 $\langle T_i, \text{BEGIN} \rangle$ 为止

证明

日志记录	事实
$\langle T, \text{BEGIN} \rangle$	
...	
$\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$	$T \in \{T_1, T_2, \dots, T_n\}$
...	
$\langle \text{END CHECKPOINT} \rangle$	T 所做的部分修改可能 已写入磁盘，必须撤销

涉及模糊检查点的故障恢复—Redo 阶段

缓冲池策略: STEAL + NO-FORCE

例 (涉及检查点的故障恢复—Redo 阶段)

日志记录	故障恢复的动作
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	
$\langle T_3, B, 10, 20 \rangle$	
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	从这里开始重做
$\langle T_2, C, 15, 25 \rangle$	
$\langle T_3, D, 20, 30 \rangle$	
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	

涉及模糊检查点的故障恢复—Redo 阶段

缓冲池策略: STEAL + NO-FORCE

例 (涉及检查点的故障恢复—Redo 阶段)

日志记录	故障恢复的动作
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	
$\langle T_3, B, 10, 20 \rangle$	
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	从这里开始重做
$\langle T_2, C, 15, 25 \rangle$	T_2 是已提交事务，将磁盘中的 C 写为 25
$\langle T_3, D, 20, 30 \rangle$	
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	

涉及模糊检查点的故障恢复—Redo 阶段

缓冲池策略: STEAL + NO-FORCE

例 (涉及检查点的故障恢复—Redo 阶段)

日志记录	故障恢复的动作
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	
$\langle T_3, B, 10, 20 \rangle$	
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	从这里开始重做
$\langle T_2, C, 15, 25 \rangle$	T_2 是已提交事务，将磁盘中的 C 写为 25
$\langle T_3, D, 20, 30 \rangle$	T_3 是不完整事务，无需重做
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	

涉及模糊检查点的故障恢复—Redo 阶段

缓冲池策略: STEAL + NO-FORCE

例 (涉及检查点的故障恢复—Redo 阶段)

日志记录	故障恢复的动作
<T ₁ , BEGIN>	
<T ₁ , A, 5, 15>	
<T ₂ , BEGIN>	
<T ₁ , COMMIT>	
<T ₃ , BEGIN>	
<T ₃ , B, 10, 20>	
<BEGIN CHECKPOINT (T ₂ , T ₃)>	从这里开始重做
<T ₂ , C, 15, 25>	T ₂ 是已提交事务, 将磁盘中的 C 写为 25
<T ₃ , D, 20, 30>	T ₃ 是不完整事务, 无需重做
<END CHECKPOINT>	
<T ₂ , COMMIT>	T ₂ 重做完毕

涉及模糊检查点的故障恢复—Undo 阶段

缓冲池策略: STEAL + NO-FORCE

例 (涉及检查点的故障恢复—Undo 阶段)

日志记录	故障恢复的动作
<T ₁ , BEGIN>	
<T ₁ , A, 5, 15>	
<T ₂ , BEGIN>	
<T ₁ , COMMIT>	
<T ₃ , BEGIN>	
<T ₃ , B, 10, 20>	
<BEGIN CHECKPOINT (T ₂ , T ₃)>	
<T ₂ , C, 15, 25>	
<T ₃ , D, 20, 30>	
<END CHECKPOINT>	
<T ₂ , COMMIT>	T ₂ 是已提交事务, 无需撤销

涉及模糊检查点的故障恢复—Undo 阶段

缓冲池策略: STEAL + NO-FORCE

例 (涉及检查点的故障恢复—Undo 阶段)

日志记录	故障恢复的动作
<T ₁ , BEGIN>	
<T ₁ , A, 5, 15>	
<T ₂ , BEGIN>	
<T ₁ , COMMIT>	
<T ₃ , BEGIN>	
<T ₃ , B, 10, 20>	
<BEGIN CHECKPOINT (T ₂ , T ₃)>	
<T ₂ , C, 15, 25>	
<T ₃ , D, 20, 30>	T ₃ 是不完整事务, 将磁盘中的 D 写为 20
<END CHECKPOINT>	
<T ₂ , COMMIT>	T ₂ 是已提交事务, 无需撤销

涉及模糊检查点的故障恢复—Undo 阶段

缓冲池策略: STEAL + NO-FORCE

例 (涉及检查点的故障恢复—Undo 阶段)

日志记录	故障恢复的动作
<T ₁ , BEGIN>	
<T ₁ , A, 5, 15>	
<T ₂ , BEGIN>	
<T ₁ , COMMIT>	
<T ₃ , BEGIN>	
<T ₃ , B, 10, 20>	
<BEGIN CHECKPOINT (T ₂ , T ₃)>	
<T ₂ , C, 15, 25>	T ₂ 是已提交事务, 无需撤销
<T ₃ , D, 20, 30>	T ₃ 是不完整事务, 将磁盘中的 D 写为 20
<END CHECKPOINT>	
<T ₂ , COMMIT>	T ₂ 是已提交事务, 无需撤销

涉及模糊检查点的故障恢复—Undo 阶段

缓冲池策略: STEAL + NO-FORCE

例 (涉及检查点的故障恢复—Undo 阶段)

日志记录	故障恢复的动作
<code><T₁, BEGIN></code>	
<code><T₁, A, 5, 15></code>	
<code><T₂, BEGIN></code>	
<code><T₁, COMMIT></code>	
<code><T₃, BEGIN></code>	
<code><T₃, B, 10, 20></code>	T₃ 是不完整事务, 将磁盘中的 B 写为 10
<code><BEGIN CHECKPOINT (T₂, T₃)></code>	
<code><T₂, C, 15, 25></code>	T ₂ 是已提交事务, 无需撤销
<code><T₃, D, 20, 30></code>	T ₃ 是不完整事务, 将磁盘中的 D 写为 20
<code><END CHECKPOINT></code>	
<code><T₂, COMMIT></code>	T ₂ 是已提交事务, 无需撤销

涉及模糊检查点的故障恢复—Undo 阶段

缓冲池策略: STEAL + NO-FORCE

例 (涉及检查点的故障恢复—Undo 阶段)

日志记录	故障恢复的动作
<code><T₁, BEGIN></code>	
<code><T₁, A, 5, 15></code>	
<code><T₂, BEGIN></code>	
<code><T₁, COMMIT></code>	
<code><T₃, BEGIN></code>	T₃ 撤销完毕, 向日志末尾写入 <code><T₃, ABORT></code>
<code><T₃, B, 10, 20></code>	T ₃ 是不完整事务, 将磁盘中的 B 写为 10
<code><BEGIN CHECKPOINT (T₂, T₃)></code>	
<code><T₂, C, 15, 25></code>	T ₂ 是已提交事务, 无需撤销
<code><T₃, D, 20, 30></code>	T ₃ 是不完整事务, 将磁盘中的 D 写为 20
<code><END CHECKPOINT></code>	
<code><T₂, COMMIT></code>	T ₂ 是已提交事务, 无需撤销

FORCE 策略下模糊检查点的结束条件

如果采用 FORCE 策略, 为何要等到 $T_1, T_2, \dots, T_n$ 全部提交后, 才能结束检查点?

- 防止检查点工作落实不到位
- 在 STEAL + FORCE 策略下, 设一个很长的不完整事务 T_1 通过了很多检查点
- 为了能撤销 T_1 , 必须保留 `<T1, BEGIN>` 之后所有的日志记录, 则检查点失去意义

```
<T1, BEGIN>
...
<BEGIN CHECKPOINT (T1, T2)>
...
<END CHECKPOINT>
...
<BEGIN CHECKPOINT (T1, T3)>
...
<END CHECKPOINT>
...
<BEGIN CHECKPOINT (T1, Tn)>
...
<END CHECKPOINT>
```

总结

① 10.1 故障

② 10.2 缓冲池策略

③ 10.3 预写式日志

- 10.3.1 预写式日志的基本概念
- 10.3.1 基于 Undo 日志的故障恢复
- 10.3.2 基于 Redo 日志的故障恢复
- 10.3.1 基于 Undo/Redo 日志的故障恢复

④ 10.4 检查点